PROJECT DOCUMENTATION / 25 OCTOBER 2026

Amzad Food

Food storefront website | Current implementation and technical handoff

A responsive food storefront built with Next.js, React and TypeScript. It presents products and articles, supports a browser-based cart, and demonstrates checkout, order history and tracking flows.

Current delivery status

Frontend prototype. The repository has no implemented backend API, database, real authentication or admin dashboard. Checkout saves an order preview on the current device and asks the customer to contact the team on WhatsApp for confirmation.

Document purpose

This document explains what exists, how to run and publish it, and what remains before a production ecommerce launch. It reflects source review at repository commit **f0e653d**. It is not a commercial scope agreement or a production acceptance certificate.

Contents

1. Website features and customer journey
2. Architecture and project structure
3. Setup, checks and deployment
4. Backend roadmap and acceptance

1. Website features and customer journey

Area	Current implementation
Storefront and catalog	Local product data; product browsing, search/filter UI, category navigation, quick view and product detail pages.
Cart and checkout	Browser-persisted cart; validated stored records; quantity capped at 20 per line; customer/address fields; promo and delivery estimates; device-only order preview.
Orders and tracking	Up to 20 saved orders in browser storage; order history/status views; labeled sample previews; downloadable HTML invoice representation.
Content and support	Blog listing/detail pages, FAQ, scripted chat, WhatsApp feedback, mobile contact controls and sample prayer schedules.
Accounts and integrations	Authentication and newsletter services are unavailable. Online payment options are disabled. No persistent shared reviews, inventory service or admin system.

Customer journey

Browse products → open details or Quick View → add items → review cart → complete checkout fields → save the local preview → contact the team on WhatsApp to request confirmation. Saving a preview does not submit an order to a server.

Rules currently encoded in the UI

Currency is BDT. Delivery costs BDT 60 and becomes automatically free when the merchandise subtotal reaches BDT 1,000 before promo discounts. WELCOME10 applies a 10% discount; AMZAD50 applies BDT 50 off from BDT 500. These are implementation values requiring business approval.

Cash on Delivery is the enabled preview payment choice. bKash, Nagad and card payments are marked coming soon. There is no payment gateway transaction or payment verification.

2. Architecture and project structure

The Next.js App Router builds a static website. React client components handle interactive behavior, while TypeScript modules supply catalog/content and browser persistence helpers. There are no implemented server-side application services.

Path	Purpose
app/page.tsx and app/layout.tsx	Home entry point and shared application layout.
app/components/	Storefront, product detail/quick view, menus, cart-related UI, account modal, history/tracking, reviews, chat and support components.
app/checkout/page.tsx	Active checkout and local preview flow. The .orig file is an old snapshot, not the active route.
app/products/ and app/blogs/	List pages and build-time dynamic detail routes.
app/lib/	Products, blogs, cart/order persistence, offers, order statuses, invoice generation and shared UI helpers.
app/globals.css and public/	Styles and static images/assets.
next.config.mjs	Static export, deployment base path, trailing slash and unoptimized images.
scripts/ and .github/workflows/	Frontend regression checks, exported-link validation and GitHub Pages publishing.
docs/	Frontend audit, backend handoff, proposed API contract and existing commercial scope material.

Data and persistence

Catalog and blog content come from local TypeScript data. Cart records use localStorage key **amzad-cart**; order records use **amzad-orders**. Customer checkout fields may remain under **af_checkout_fields**. Device storage is not an account, shared database, stock system or proof of order/payment.

Cart identity currently uses product names. Backend integration should introduce immutable product and variant IDs, preserve historical order snapshots, and define migration or reset behavior for existing browser records.

3. Setup, checks and deployment

Local development

Use Node.js 20 to match the CI configuration, with npm. From the project root:

```
npm ci
npm run dev
```

Open **http://localhost:3001/amzad-food-website/**. The development script uses port 3001. If needed, bind to localhost with `npm run dev -- --hostname 127.0.0.1`. No environment variables are currently required for the frontend demonstration.

Verification commands

```
npx tsc --noEmit
node scripts/check-frontend.cjs
npm run build
python3 scripts/check-static-export.py
```

The frontend check covers storage, quantity, order and shipping regressions. The export checker validates local HTML asset/link targets; it does not verify external URLs, all client interactions or accessibility. The lint script exists but ESLint is not configured as an unattended verification gate.

Build and hosting

Production output is **out/**; build intermediates use **.next/**, while development uses **.next-dev/**. The current static export is not served using `npm run start`. Publish the exported files through an appropriate static host.

The existing GitHub Actions workflow installs dependencies, builds and deploys out/ to GitHub Pages on pushes to main or manual dispatch. The configured base path is **/amzad-food-website**, with trailing slashes and unoptimized images.

A new domain or root-path deployment requires reviewing hardcoded asset and route prefixes across TSX, data and CSS. A runtime Next.js deployment also requires an intentional migration away from static export and a replacement publishing workflow.

Evidence and limits

The existing 24 October audit records passing TypeScript, regression checks and production build after frontend repairs, plus 83 exported HTML files with zero missing local targets. Those are prior recorded results, not checks rerun for this PDF. Browser layout, keyboard interaction, live integrations and production security remain unverified.

4. Backend roadmap and acceptance

Recommended implementation order

1. Agree the business rules. Approve sellable catalog, variants, pricing, policies, marketing claims, shipping, payments, hosting and ownership. Review docs/API_CONTRACT.md as a proposal, not an implemented service.

2. Connect catalog and identity. Add database-backed products/variants and API adapters. Replace name-based purchase identity; define product publishing and route-generation behavior.

3. Implement accounts and guest access. Add real authentication, session lifecycle and recovery if retained. Protect order history, customer details and invoices with an owning session or verified guest access.

4. Implement authoritative checkout. Quote prices, discounts, delivery and stock on the server. Persist orders atomically, use idempotency for retries, and clear the cart only after confirmed success.

5. Complete operations. Connect history, tracking, invoices, approved reviews and subscriptions. Implement separately agreed admin workflows, monitoring, backups and deployment procedures.

Production acceptance checklist

Confirm mobile/desktop navigation, direct nested routes, catalog images, cart and checkout on the final host. Verify exactly one persistent order after retries or double clicks; failed requests must preserve the cart. Test unauthorized order/invoice access, changed prices, concurrent stock changes, expired sessions and genuine empty/error states.

Confirm admin status updates reach customer views and historical invoice totals survive catalog changes. Approve customer-facing policies and remove production sample fallbacks. Record staging evidence, final acceptance, backup/restore steps and rollback instructions.

Maintainer references

Repository document	Use
README.md	Local setup, current limitations and primary commands.
docs/FRONTEND_AUDIT.md	Original findings, repair updates, evidence and remaining verification.
docs/BACKEND_HANDOFF.md	Integration replacement points, risks and delivery checklist.
docs/API_CONTRACT.md	Proposed resources, money/timestamp conventions and order access model.

Read repair updates before older finding tables: some handoff descriptions intentionally retain the original baseline. Current code and the latest repair record take precedence when evaluating completed frontend repairs.